

Integ aplicaciones computación

Otoño 2026

Aplicación Python TK y consumo de microservicios

Fabian Azaed Orta Singlaterry 613504

Código de honor

Doy mi palabra que he realizado esta actividad con integridad académica.

Conclusión y Reflexión Profesional

Explique, a partir de su propia implementación:

Qué parte de la integración presentó mayor dificultad.

Qué problema encontró durante el desarrollo.

Cómo identificó la causa.

Qué modificación realizó para resolverlo.

Qué ocurre en su aplicación cuando un microservicio deja de responder.

Qué diferencia observó entre ejecutar los servicios localmente y consumirlos remotamente.

Qué aprendió sobre la relación entre cliente, microservicio y base de datos

Más que todo, la mayor dificultad fue leer todas las instrucciones y analizar por posibles errores para pasarlo a un agente de IA que pueda hacer el código. Note algunas cosas como que en las instrucciones venía que en un endpoint se buscara isbn con “Nombre” + “ID Estudiante” por lo que iba dar error pq el isbn es el código de 10 números del libro y tiene un formato. Ya leyendo el código fui quitando algunas cosas y definiendo otras y ya quedo un master prompt que me sirvió para implementar la app con DeepSeek Flash 4.1. Aca hay algunas notas extra.

La parte que más me costó fue **la sesión**. El microservicio de login no usa cookies: devuelve un token que hay que enviar en la cabecera `X-Session-Token` en cada petición. Además, `/login` y `/register` exigen resolver un CAPTCHA, algo que el enunciado no menciona.

El problema más difícil fue un **403 al cambiar la contraseña** con la actual correcta. Comprobé el hash en la base con `bcrypt.checkpw` y era válido, así que el fallo estaba en mi cliente: `update_profile` sobrescribía `current_password` con una cadena vacía aunque ya viniera en los cambios. El servicio recibía una contraseña en blanco y la rechazaba; lo corregí para que solo la establezca cuando falta.

También apareció un fallo de **hilos**: llamaba a `after()` desde el hilo trabajador y Tk no lo admite. Lo resolví con una cola que el hilo principal vacía en su bucle.

Cuando un microservicio deja de responder, la aplicación **no se cierra ni se congela**: su semáforo pasa a rojo, el aviso explica qué pasó y el resto de pantallas siguen funcionando. Al restaurarlo, vuelve a verde solo.

Entre local y remoto no cambia **nada del programa**: solo la dirección configurada. En local la respuesta es inmediata; en la instancia se nota la latencia de red, por eso los tiempos de espera son configurables.

Lo que más claro me quedó es la separación de responsabilidades: el cliente nunca toca la base de datos, solo habla HTTP con el microservicio, y es este quien decide qué hacer con PostgreSQL. Por eso pude apagar el servicio de libros y la aplicación siguió en pie.